

Module 2

Verilator Deep Dive

Learning objectives

- Verilator Fundamentals: Understanding how Verilator converts Verilog to C++
- Compilation Process: Mastering Verilator compilation flags and options
- C++ Testbench Writing: Creating effective testbenches using C++ and Verilator's API
- Waveform Generation: Creating VCD/FST files for debugging
- File I/O: Managing test vectors and results with files
- Debugging Techniques: Strategies for debugging Verilator testbenches

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["Verilator Overview"]

T2["Compilation Process"]

T1 --> T2

T3["C++ Testbench Structure"]

T2 --> T3

Master Verilator for C++ testbench development

Overview

- This module provides comprehensive coverage of Verilator, a fast Verilog/SystemVerilog simulator th...

Design architecture

RTL / block architecture

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart LR

VER[verilator --cc] --> OBJ[obj_dir]

subgraph dut2["module2/dut"]

M4[mux_4to1]

CTR[counter_4bit]

end

Module 2: DUT / block hierarchy (see module2/examples/).

Verification environment architecture

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

LOOP[cycle eval loop] --> TRACE[optional trace]

TRACE --> FILE[file-driven C++ tests]

Module 2: UVM layers, agents, and checkers for this phase.

1. Verilator compilation model

- Input: Verilog/SystemVerilog RTL + optional
`--timing` / lint flags per example
- Output: C++ classes (`V<top>`) in `obj\_dir/` with
fast cycle-accurate `eval()`
- Linkage: C++ `main()` or test class calls
`dut->eval()` each cycle or on events
- Tracing: `--trace` / `--trace-fst` for waveform
dumps from the C++ side

2. Module 2 DUT catalog

- ``dut/multiplexers/``: Same MUX family as Module 1 — cross-tool comparison baseline
- ``dut/counters/``: ``counter_4bit`` — exercises sequential C++ drive/monitor loops
- Port mapping: Verilator flattens ports to struct members (``dut->clk``, ``dut->count``)
- Performance: Large test suites favor Verilator's compiled model over interpreted VVP

3. C++ testbench architecture

- Top-level ``main``: Allocates ``Vtop``, applies reset sequence, runs clock loop
- Stimulus loop: ``for`/`while`` sets inputs then ``eval()`` advances time
- Optional OOP: Advanced tests organize Driver/Monitor/Checker as C++ classes
- Build: Generated ``Vtop.mk`` plus user ``Makefile`` link testbench and model

Verification & testing methods

Testing and closure flow

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TD

A[cycle tests] --> B[trace debug]

B --> C[batch module2.sh]

Module 2: how tests, checks, and metrics close verification goals.

1. Cycle-based C++ testing

- Clock generation: Toggle clock in loop with ``eval()`` between edges
- Reset sequence: Hold ``rst_n`` low N cycles, release, verify known state
- Checking: ``assert()`` or explicit compare with ``std::cerr`` on mismatch

2. Tracing and performance checks

- Waveforms: Enable Verilator trace API; view in GTKWave for debug examples
- Printf debug: Structured logging before/after ``eval()`` for signal snapshots
- Regression: `./scripts/module2.sh --all-tests``
batches compile+run across examples

3. File I/O and scalable stimulus

- Vector files: Read test data in C++ (``file_io`` examples) for long sequences
- Repeatability: Fixed seeds and deterministic loops for comparable runs
- Failure isolation: Run single ``obj_dir`` binary with minimal stimulus to bisect bugs

Syllabus topics

1. Verilator Overview (1/6)

- What is Verilator?
- Fast Verilog/SystemVerilog simulator
- Generates C++ code from Verilog
- Open-source and free
- Excellent performance
- Verilator's Compilation Model

1. Verilator Overview (2/6)

- Two-stage compilation
- Verilog → C++ conversion
- C++ compilation and linking
- Generated wrapper classes
- C++ Wrapper Generation
- Top module class structure

1. Verilator Overview (3/6)

- Signal access methods
- Evaluation methods
- Tracing support
- Verilator Capabilities and Features
- Fast simulation
- Good Verilog-2001 support

1. Verilator Overview (4/6)

- Limited SystemVerilog support
- Coverage support
- Waveform generation
- Supported Verilog/SystemVerilog Constructs
- Most Verilog-2001 features
- Basic SystemVerilog features

1. Verilator Overview (5/6)

- Interfaces (limited)
- Classes (limited)
- Assertions (limited)
- Limitations and Workarounds
- Limited SystemVerilog support
- Some advanced features not supported

1. Verilator Overview (6/6)

- Workarounds for unsupported features
- When to Use Verilator
- High-performance simulation
- Large designs
- Integration with C++ libraries
- Complex testbenches with advanced data structures

2. Compilation Process (1/8)

- Verilator Compilation Command
- Basic syntax: ``verilator --cc --exe design.v
testbench.cpp``
- Output generation
- Build process
- Compilation Flags and Options
- ``--cc``: Generate C++ code

2. Compilation Process (2/8)

- ``--exe``: Generate executable
- ``--build``: Build automatically
- ``-I<directory>``: Add include path
- ``-D<macro>``: Define macro
- ``-O0, -O1, -O2, -O3``: Optimization levels
- ``--trace``: Generate VCD tracing

2. Compilation Process (3/8)

- ``--trace-fst`` : Generate FST tracing
- ``--coverage`` : Enable coverage
- ``--lint-only`` : Lint without compilation
- Optimization Levels (-O0, -O1, -O2, -O3)
- ``-O0`` : No optimization (fastest compilation)
- ``-O1`` : Basic optimization

2. Compilation Process (4/8)

- ``-O2``: More optimization (default)
- ``-O3``: Maximum optimization (slowest compilation, fastest simulation)
- Coverage Options (`--coverage`)
- Code coverage generation
- Coverage analysis
- Coverage reporting

2. Compilation Process (5/8)

- Linting Capabilities (--lint-only)
- Syntax checking
- Design rule checking
- No simulation generation
- Waveform Generation (--trace, --trace-fst)
- `--trace`: Generate VCD files

2. Compilation Process (6/8)

- `--trace-fst`: Generate FST files (smaller, faster)
- Signal selection
- GTKWave compatibility
- Include Path Management
- Adding include directories
- Multiple include paths

2. Compilation Process (7/8)

- Relative vs. absolute paths
- Define Macros
- Conditional compilation
- Macro definitions
- Usage in designs
- Multi-File Compilation

2. Compilation Process (8/8)

- Compiling multiple files
- File ordering
- Dependency management
- ``basic_compilation.cpp``: Demonstrates basic compilation, optimization, tracing, coverage, linting

3. C++ Testbench Structure (1/7)

- C++ Main Function
- Entry point
- Verilator initialization
- DUT instantiation
- Test sequence
- Cleanup

3. C++ Testbench Structure (2/7)

- Verilator-Generated Class Usage
- Class naming (V<module_name>)
- Instance creation
- Method calls
- Signal access
- DUT Instantiation in C++

3. C++ Testbench Structure (3/7)

- Creating DUT object
- Memory management
- Lifetime management
- Signal Access (Reading and Writing)
- Reading signals: ``dut->signal_name``
- Writing signals: ``dut->signal_name = value``

3. C++ Testbench Structure (4/7)

- Signal types
- Multi-bit signals
- Clock Generation in C++
- Simulation loop
- Clock toggling
- Clock period control

3. C++ Testbench Structure (5/7)

- Multiple clocks
- Reset Generation in C++
- Reset sequences
- Synchronous reset
- Asynchronous reset
- Reset timing

3. C++ Testbench Structure (6/7)

- Simulation Loop
- Time management
- Event scheduling
- Loop structure
- Termination
- ``mux_4to1_test.cpp``: Basic C++ testbench structure

3. C++ Testbench Structure (7/7)

- ``counter_test.cpp``: Clock and reset generation, simulation loop

4. C++ Testbench Writing (1/6)

- Basic C++ Testbench Template
- Standard structure
- Common patterns
- Best practices
- Signal Access Methods
- Direct access

4. C++ Testbench Writing (2/6)

- Signal reading
- Signal writing
- Signal monitoring
- Clock and Reset Patterns
- Standard clock patterns
- Reset patterns

4. C++ Testbench Writing (3/6)

- Clock/reset coordination
- Stimulus Generation in C++
- Test vector generation
- Pattern generation
- Sequential stimulus
- Random stimulus

4. C++ Testbench Writing (4/6)

- Response Monitoring in C++
- Output monitoring
- Real-time monitoring
- Post-processing
- Logging
- Result Checking

4. C++ Testbench Writing (5/6)

- Expected value calculation
- Output comparison
- Error detection
- Assertions
- Error Reporting
- Error messages

4. C++ Testbench Writing (6/6)

- Exit codes
- Test summaries
- Failure reporting

5. Verilator C++ API (1/5)

- Top Module Class Structure
- Class name: ``V<module_name>``
- Constructor/destructor
- Methods
- Signals
- Signal Access Methods

5. Verilator C++ API (2/5)

- Direct member access
- Signal types
- Multi-bit signals
- Array signals
- Evaluation Method (eval())
- What eval() does

5. Verilator C++ API (3/5)

- When to call eval()
- Timing considerations
- Multiple evaluations
- Time Management
- Simulation time
- Time tracking

5. Verilator C++ API (4/5)

- Time advancement
- Time-based events
- Tracing API
- VerilatedVcdC class
- trace() method
- dump() method

5. Verilator C++ API (5/5)

- File management
- Coverage API
- Coverage collection
- Coverage reporting
- Coverage analysis

6. Waveform Generation (1/5)

- VCD File Generation
- Using --trace flag
- VerilatedVcdC class
- trace() method
- dump() method
- File management

6. Waveform Generation (2/5)

- FST File Generation
- Using --trace-fst flag
- FST advantages
- GTKWave compatibility
- Signal Selection for Tracing
- trace() method levels

6. Waveform Generation (3/5)

- Selective tracing
- Hierarchy levels
- GTKWave Compatibility
- VCD format
- FST format
- GTKWave viewing

6. Waveform Generation (4/5)

- Signal organization
- Waveform Analysis
- Viewing waveforms
- Signal timing
- Timing relationships
- Debugging with waveforms

6. Waveform Generation (5/5)

- ``waveform_example.cpp``: Comprehensive waveform generation using Verilator tracing API

7. Debugging with Verilator (1/6)

- Compilation Error Debugging
- Understanding Verilator errors
- Common compilation errors
- Syntax errors
- Missing files
- C++ Compilation Errors

7. Debugging with Verilator (2/6)

- Understanding C++ errors
- Linker errors
- Template errors
- Common mistakes
- Runtime Debugging
- Understanding runtime errors

7. Debugging with Verilator (3/6)

- Signal inspection
- GDB debugging
- Valgrind debugging
- Signal Inspection
- Printing signal values
- Signal monitoring

7. Debugging with Verilator (4/6)

- VCD file analysis
- Debug output
- Logging Strategies
- Debug levels
- Conditional logging
- File-based logging

7. Debugging with Verilator (5/6)

- Log organization
- Common Pitfalls and Solutions
- Forgetting to call eval()
- Signal initialization
- Memory leaks
- Timing issues

7. Debugging with Verilator (6/6)

- Common mistakes
- ``debug_example.cpp``: C++ debugging techniques, logging strategies, error handling

8. Advanced Verilator Features (1/4)

- Multi-Threaded Simulation Basics
- Thread safety
- Parallel simulation
- Synchronization
- Performance Optimization
- Compilation optimization

8. Advanced Verilator Features (2/4)

- Simulation optimization
- Memory optimization
- Speed optimization
- Large Design Handling
- Compiling large designs
- Memory management

8. Advanced Verilator Features (3/4)

- Incremental compilation
- Design partitioning
- Custom C++ Integration
- Integrating C++ libraries
- External interfaces
- Custom functions

8. Advanced Verilator Features (4/4)

- System integration
- SystemC Integration Basics
- SystemC support
- SystemC integration
- When to use SystemC

9. Project Organization (1/7)

- Makefile Integration
- Basic Makefile structure
- Verilator compilation rules
- Test execution
- Clean targets
- Dependency management

9. Project Organization (2/7)

- CMake Integration
- CMake setup
- Verilator integration
- Build configuration
- Scripting for Automation
- Shell scripts for automation

9. Project Organization (3/7)

- Batch processing
- Test automation
- Result collection
- Batch Simulation
- Running multiple tests
- Test selection

9. Project Organization (4/7)

- Parallel execution
- Result aggregation
- Regression Testing Setup
- Test suite organization
- Regression test structure
- Pass/fail reporting

9. Project Organization (5/7)

- Test result tracking
- Location: ``module0/examples/verilator_basics/and_gate_test.cpp``
- Demonstrates: Basic C++ testbench structure, signal access
- Location:
``module2/examples/cpp_testbench/mux_4to1_test.cpp``
- Demonstrates: Complete C++ testbench, signal access, test patterns
- Location:
``module2/examples/cpp_testbench/counter_test.cpp``

9. Project Organization (6/7)

- Demonstrates: Clock generation, reset sequences, simulation loop
- Location:
 `module2/examples/file\_io/file\_io\_test.cpp`
- Demonstrates: Reading test vectors from files, writing results
- Location: See compilation examples
- Demonstrates: Multi-file compilation, include paths
- Location:
 `module2/tests/basic\_tests/test\_mux\_4to1.cpp`

9. Project Organization (7/7)

- Demonstrates: Class-based organization, comprehensive testing

Hands-on examples

Module 2 self-check

```
$ ./scripts/module2.sh --check
```

Terminal: module2_check

```
=====
Module 2: environment self-check
=====
[OK] module2/ present
[OK] module2/examples/ present
[INFO] verilator not installed (needed for Module 2+ labs: ./scripts/install_verilator.sh)
[OK] iverilog: Icarus Verilog version 12.0 (stable) ()
All required checks passed
```

Exercise scaffold

```
./scripts/module2.sh --scaffold
```

Demo: Compilation

```
$ head -25 module2/examples/compilation/Makefile
```

Terminal: ex01_compilation

```
# =====
# Makefile for Verilator Compilation Examples
# =====
# This Makefile demonstrates various Verilator compilation options and flags.
# Each target shows a different compilation configuration.
#
# Compilation Options Demonstrated:
# - Basic compilation (minimal flags)
# - Optimization levels (-O2)
# - Waveform tracing (--trace)
# - Code coverage (--coverage)
# - Linting only (--lint-only, no code generation)
#
# Usage:
# make all          # Run all compilation examples
# make basic_compilation # Basic compilation
# make optimization  # Compilation with optimization
# make tracing       # Compilation with waveform tracing
# make coverage      # Compilation with code coverage
# make linting       # Syntax checking only (no compilation)
# make clean         # Remove build artifacts
# =====

VERILATOR = verilator
DUT_DIR = ../dut
```

Demo: C++ testbench

```
$ ls -la module2/examples/cpp_testbench
```

Terminal: ex02_cpp_testbench

```
total 24
drwxrwxrwx 1 yongfu yongfu 4096 May 28 12:19 .
drwxrwxrwx 1 yongfu yongfu 4096 May 28 12:19 ..
-rwxrwxrwx 1 yongfu yongfu 3567 May 28 12:19 Makefile
-rwxrwxrwx 1 yongfu yongfu 8182 May 28 12:19 counter_test.cpp
-rwxrwxrwx 1 yongfu yongfu 8695 May 28 12:19 mux_4to1_test.cpp
```

Demo: File I/O

```
$ ls -la module2/examples/file_io
```

Terminal: ex03_file_io

```
total 8
drwxrwxrwx 1 yongfu yongfu 4096 May 28 12:19 .
drwxrwxrwx 1 yongfu yongfu 4096 May 28 12:19 ..
-rwxrwxrwx 1 yongfu yongfu  715 May 28 12:19 Makefile
-rwxrwxrwx 1 yongfu yongfu 7969 May 28 12:19 file_io_test.cpp
-rwxrwxrwx 1 yongfu yongfu   75 May 28 12:19 test_vectors.txt
```

Demo: Waveforms

```
$ ls -la module2/examples/waveforms
```

Terminal: ex04_waveforms

```
total 12
drwxrwxrwx 1 yongfu yongfu 4096 May 28 12:19 .
drwxrwxrwx 1 yongfu yongfu 4096 May 28 12:19 ..
-rwxrwxrwx 1 yongfu yongfu 749 May 28 12:19 Makefile
-rwxrwxrwx 1 yongfu yongfu 6034 May 28 12:19 waveform_example.cpp
```

Demo: Debugging

```
$ ls -la module2/examples/debugging
```

Terminal: ex05_debugging

```
total 8
drwxrwxrwx 1 yongfu yongfu 4096 May 28 12:19 .
drwxrwxrwx 1 yongfu yongfu 4096 May 28 12:19 ..
-rwxrwxrwx 1 yongfu yongfu 523 May 28 12:19 Makefile
-rwxrwxrwx 1 yongfu yongfu 5981 May 28 12:19 debug_example.cpp
```

Practice & assessment

Assessment checklist

- Can compile Verilog designs with various Verilator options
- Can write complete C++ testbenches
- Can use Verilator C++ API effectively
- Can use file I/O in C++ testbenches
- Can generate and view waveforms
- Can debug compilation and simulation errors

Summary & next steps

- Master Verilator for C++ testbench development
- Complete module2/CHECKLIST.md
- Run `./scripts/module2.sh --check`
- Next: Next module in course